

Generative AI: How Deep Learning Models

Professor Mary Lou Maher





Topics

- Deep learning neural networks: nodes, layers, weights, training, backpropagation
- Large Language Models phases of learning: pretraining, finetuning, prompting
- Generative AI: tokenization, transformers, training objectives
- Types of foundation models: GPT, BERT, GAN, VAE, RNN



Deep Learning Networks

Key Terms

How Deep Learning Networks Work

A deep learning network is a type of neural network made up of many connected processing units that learn patterns from data by adjusting numerical values called weights.

Deep learning networks learn by passing data through layers of nodes, measuring error, and using backpropagation to adjust weights so predictions improve over time.

Key terms for Deep Learning Networks:

- Nodes (Neurons)
- Layers
- Weights
- Training
- Backpropagation
- Learning over time



Nodes

Nodes, also called neurons, are the basic computational units of the network.

Each node:

- Receives numerical inputs
- Multiplies them by weights
- Sums the results
- Applies an activation function
- Produces an output

Nodes represent learned features of the data.

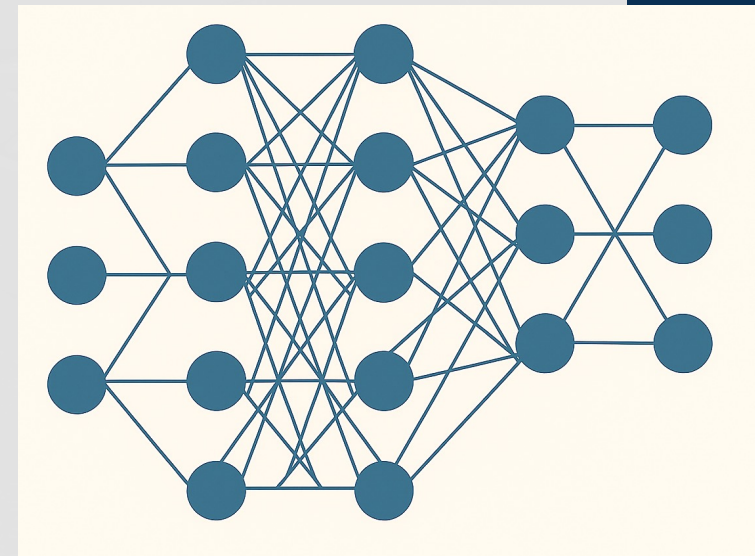


Layers

Deep learning networks are organised into layers:

- **Input layer:** Receives the raw input data (e.g. pixel values, token IDs).
- **Hidden layers:** Perform intermediate processing and feature extraction. Having many hidden layers is what makes the network “deep”.
- **Output layer:** Produces the final prediction or output.

Each layer passes information forward to the next layer.



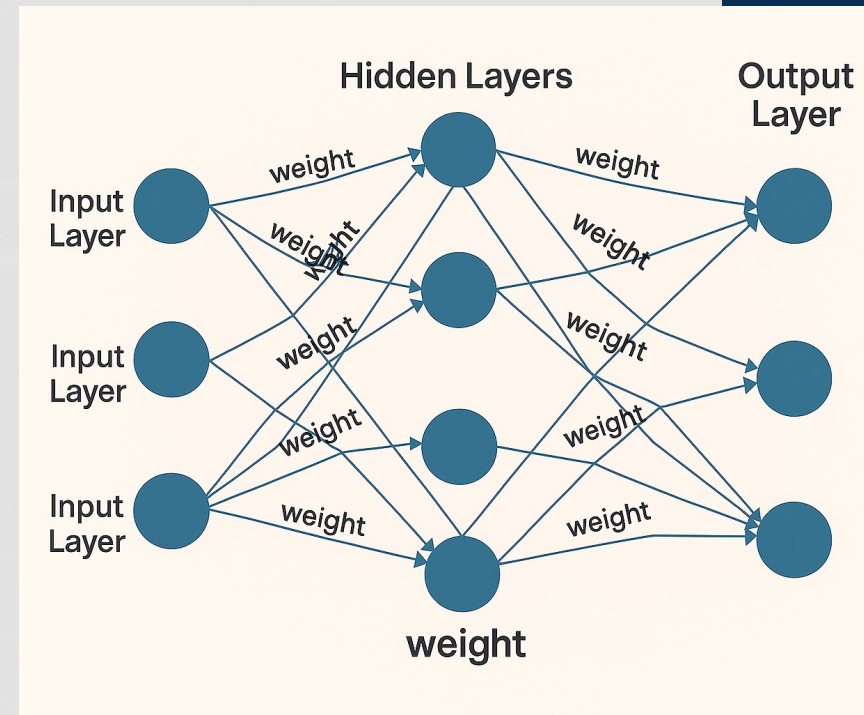
Weights

Weights are numerical values on the connections between nodes.

Weights control how strongly one node influences another.

Learning in a neural network is essentially finding better values for these weights.

Large models (like LLMs) can have millions or billions of weights.



Training

Training is the process of teaching the network to make accurate predictions.

Training involves:

- Providing input data
- Producing an output (prediction)
- Comparing the prediction to the correct answer

The goal is to minimise error, measured using a loss function.

Training happens over many examples and many repetitions.



Backpropagation

Backpropagation is the algorithm used to update the weights. It works by:

- Calculating how much error each weight contributed to the final output
- Sending this error information backwards through the network
- Using this information, each weight is adjusted slightly to reduce future error.

In simple terms:

Backpropagation tells each weight how to change to make the network better.



Learning over time

Training consists of repeated cycles of:

- Forward pass (make a prediction)
- Loss calculation (measure error)
- Backpropagation (update weights)

Over time:

- Errors decrease
- The network learns useful representations
- Predictions become more accurate

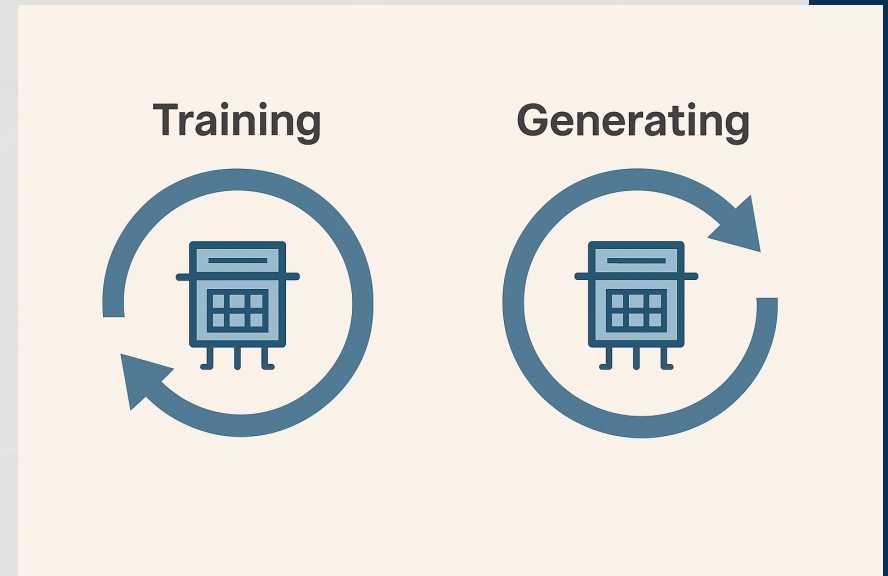


LLM Models Phases: Training and Generating



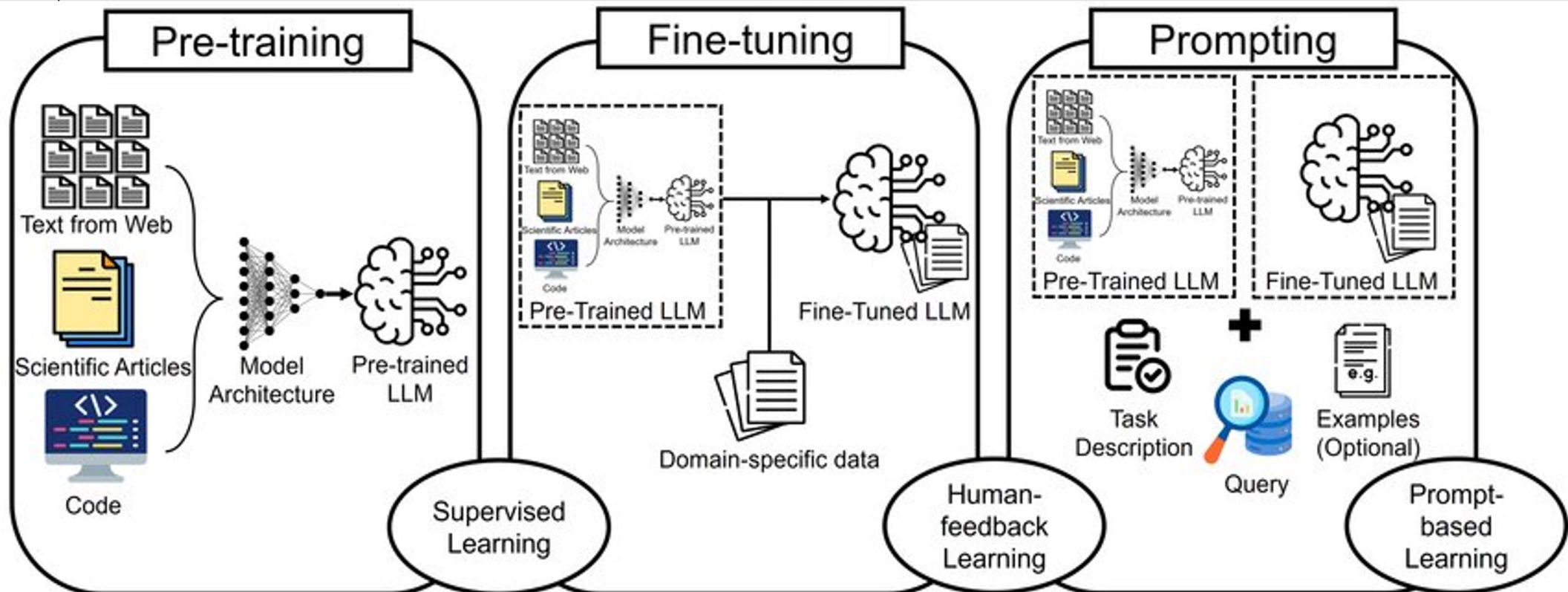
Large Language Models Work Through A Two Phase Process

- Training
 - Data collection
 - Tokenization
 - Format tokenized data into input-output pairs
 - Train a Transformer model* to predict output from input
 - Finetune using specific tasks
- Generating
 - Tokenize input text
 - Predict the next tokens
 - Sample a token from this distribution
 - Append token and repeat
 - Continue until a stopping criterion



* This is the model used by ChatGPT





Ghali, Mohammed-Khalil & Farrag, Abdelrahman & Sakai, Hajar & El Baz, Hicham & Jin, Yu & Lam, Sarah. (2024). GAMEDX: Generative AI-based Medical Entity Data Extractor Using Large Language Models. 10.48550/arXiv.2405.20585.

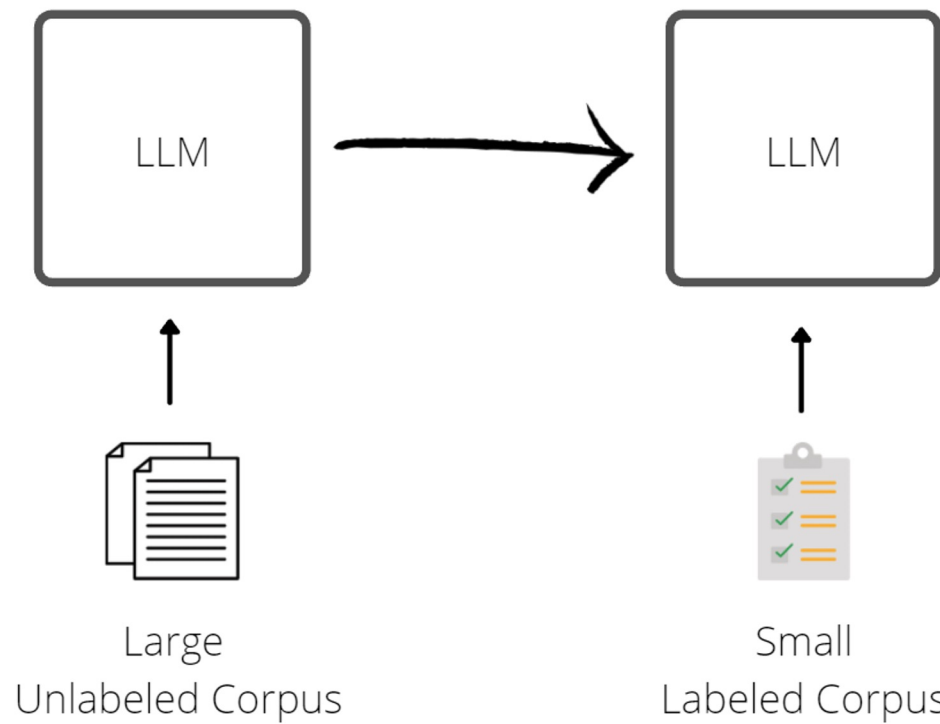




LLM Pretraining

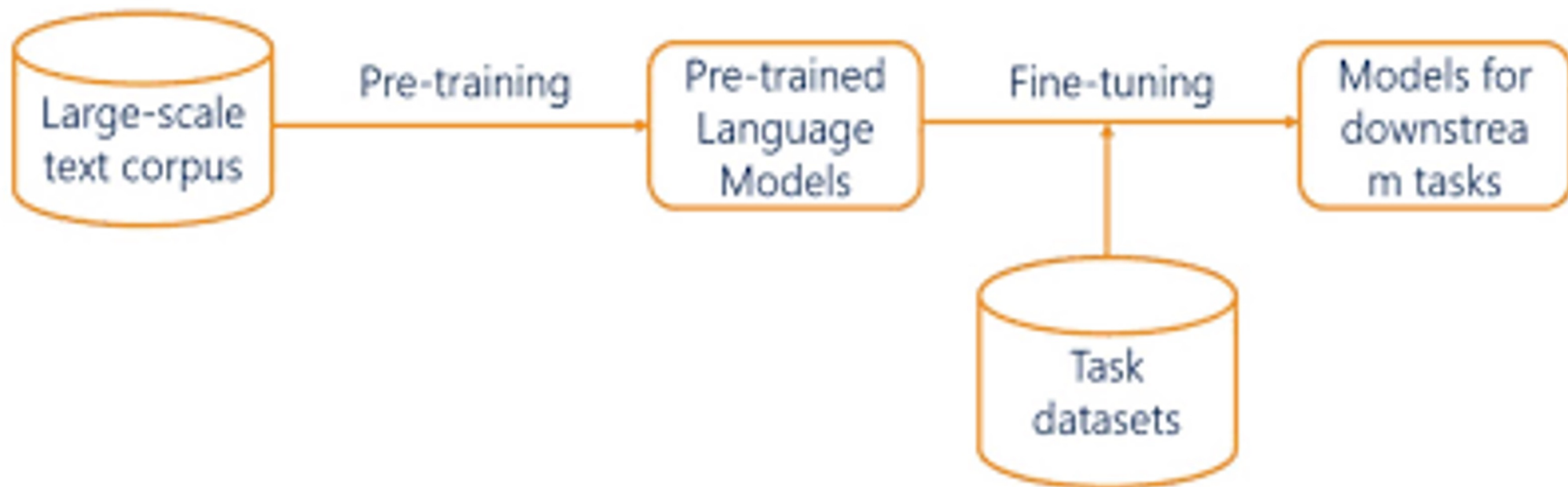
Pre-Training
(Computationally Expensive)

Fine-Tuning
(Cheaper)





Training a Large Language Model



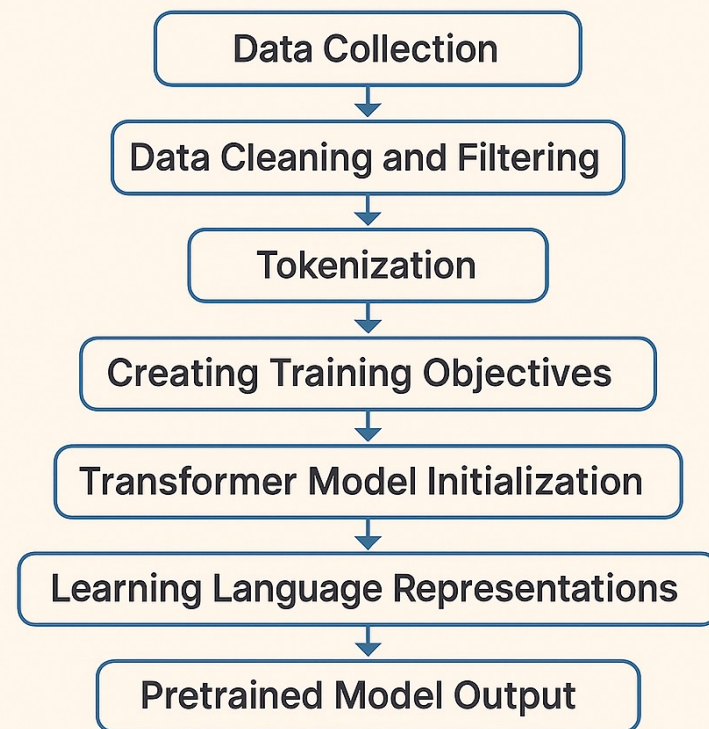
<https://www.microsoft.com/en-us/research/uploads/prod/2021/06/Pre-training-Models-Xu-Tan.pdf>

LLM Pretraining Phase

Pretraining is the phase where a model learns general language patterns from massive amounts of text, before being adapted to specific tasks.

1. Data Collection
2. Data Cleaning and Filtering
3. Tokenization
4. Creating Training Objectives
5. Transformer Model Initialisation
6. Training with Backpropagation
7. Learning Language Representations
8. Pretrained Model Output

Pretraining an LLM



Pretraining: Data Collection

Goal: expose the model to a wide range of language use, styles, and domains

Gather very large, diverse text corpora

Examples:

- Web pages
- Books
- Wikipedia
- Articles and documents



Data Cleaning and Filtering

Goal: Improve training stability and model quality

Remove:

- Duplicates
- Very low-quality text
- Corrupted or non-linguistic data
- Optional filtering for:
 - Language
 - Length
 - Formatting consistency



Tokenization

Goal: convert text to a format that can be processed mathematically by the model

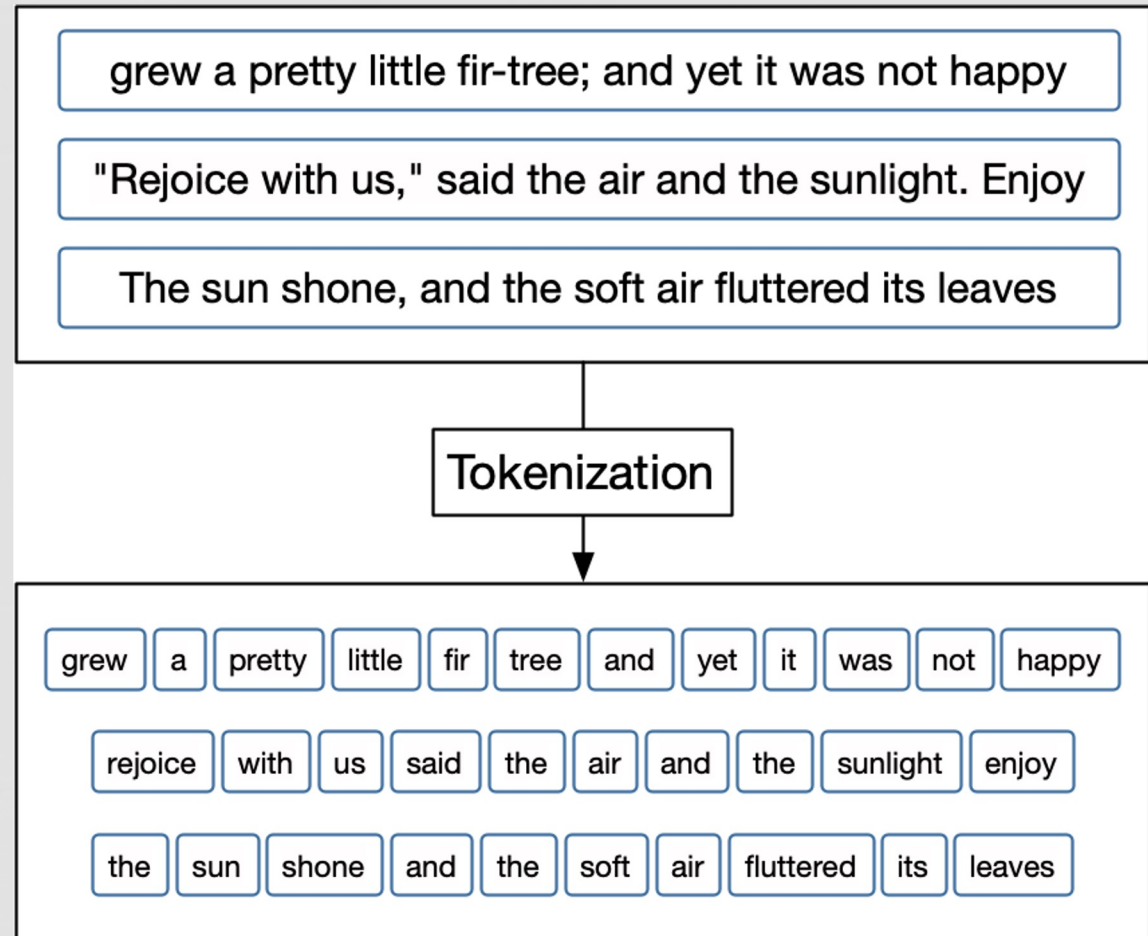
- Convert raw text into tokens
- Tokens may be words, subwords, or character sequences
- Build a vocabulary of tokens
- Each token is mapped to a numerical ID

This works particularly well because it connects:

- text → symbols → numbers



Example of Tokenization





Alternative Tokens for the same data

Sample Data:

"This is tokenizing."

Character Level

[T] [h] [i] [s] [i] [s] [t] [o] [k] [e] [n] [i] [z] [i] [n] [g] [.]

Word Level

[This] [is] [tokenizing] [.]

Subword Level

[This] [is] [token] [izing] [.]

<https://shaankhosla.substack.com/p/talking-tokenization>

Select Training Objectives

The model is trained using unsupervised learning.

Examples of training objectives:

- **Next-token prediction** (used by GPT-style models): Given previous tokens, predict the next one
- **Masked-language modelling** (used by BERT-style models): Predict missing tokens in a sentence

No human labels are required—the text itself provides the supervision.

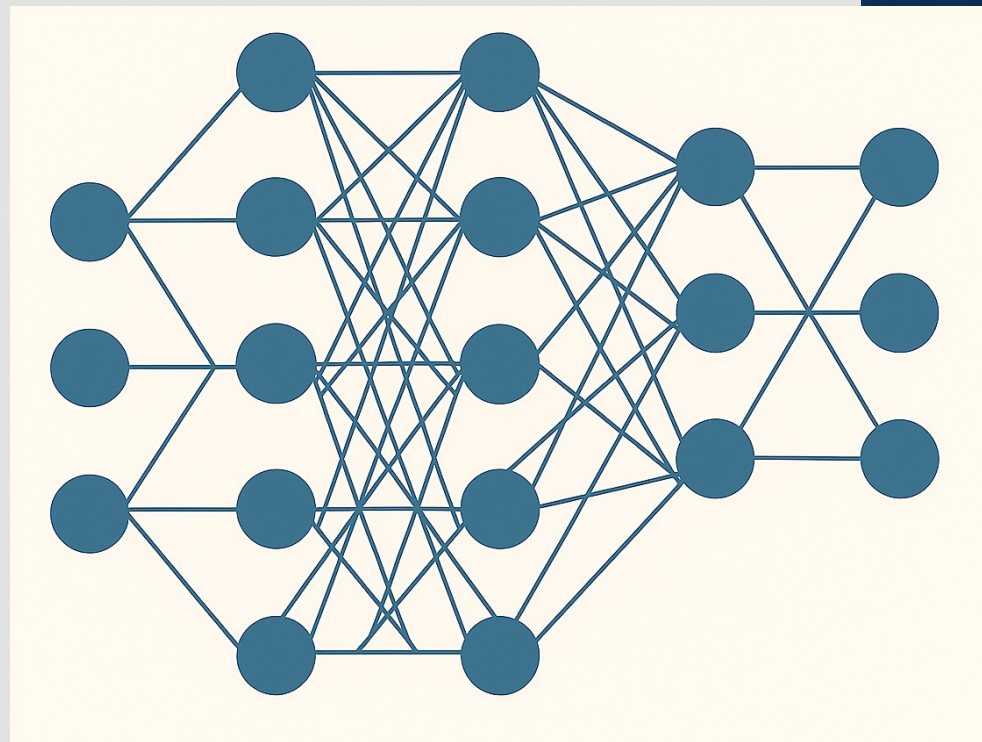


Transformer Model Initialization

Define the model architecture:

- Embedding layers
- Positional encoding
- Multi-head self-attention
- Feed-forward layers
- Initialise parameters (weights) randomly

The number of parameters can range from millions to hundreds of billions



- Layers (input → multiple hidden layers → output)
- Neurons (nodes) as computation units
- Weighted connections between layers



Training with Backpropagation

Backpropagation is the process of adjusting a neural network's weights by propagating error backward through the network.

For each batch of tokenized text:

- Tokens are passed through the Transformer
- The model predicts token probabilities
- A loss function measures prediction error
- Gradients are computed using backpropagation
- Parameters are updated using an optimizer

This is repeated over many epochs and massive datasets





When OpenAI says they trained a model on 400B parameters and then google says they trained 600B parameters, what does that mean?

- **"Parameters" is a synonym for "weights". What is a weight?**
 - The number of parameters in a language model refers to the number of weights in the model's neural network. A model with a larger number of parameters can potentially have more "capacity," meaning it potentially has the ability to fit a more complex set of patterns in the data it was trained on.
- The parameters of the LLM are the different features of the language model, such as the ability to generate different types of text, the ability to translate languages, and the ability to summarise text. The more parameters an LLM has, the more complex it can be and the more tasks it can perform.

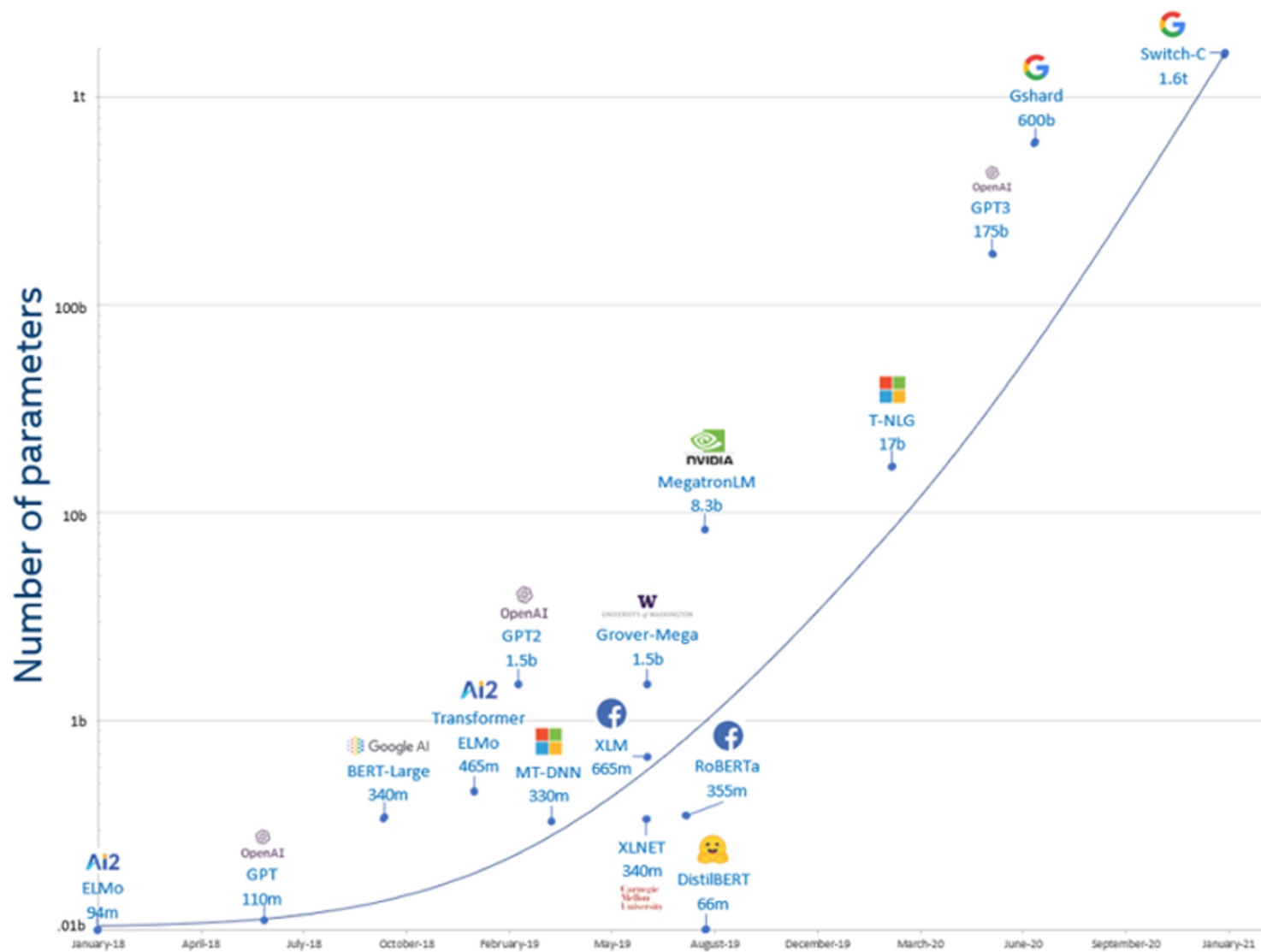


Figure 1: Exponential growth of number of parameters in DL models

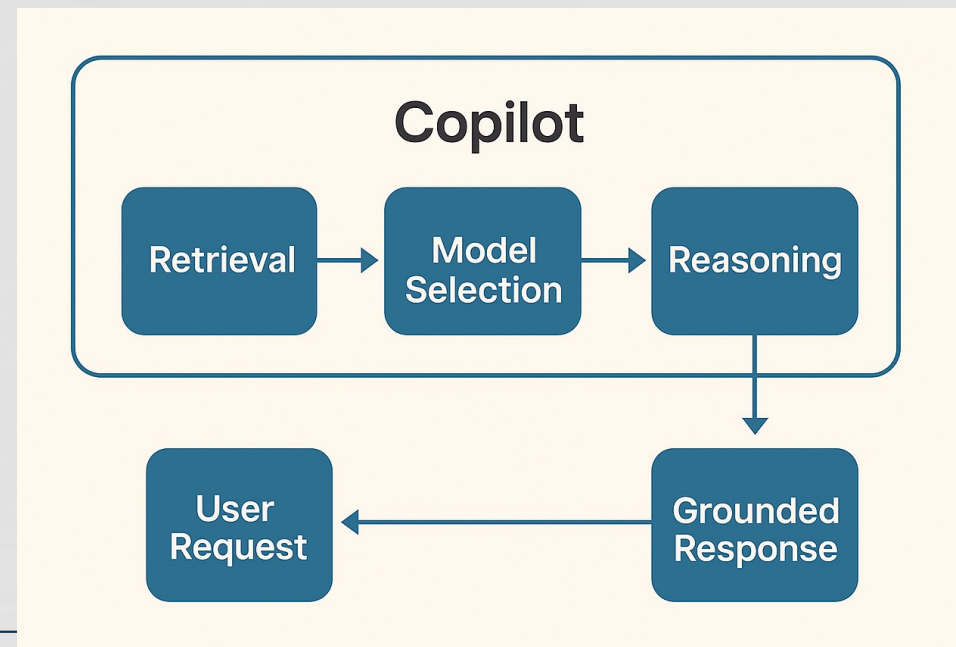


Where is Copilot on the number of parameters chart?

“Unlike GPT or BERT, Copilot isn’t just one neural network. It’s a coordination layer that retrieves data, chooses models, applies reasoning, and produces a grounded response.”

- **User Request** starts outside the model
- **Retrieval** pulls in relevant documents, emails, files, and context
- **Model Selection** dynamically chooses an appropriate GPT-class model
- **Reasoning** applies depth and multi-step inference
- **Grounded Response** is produced using retrieved enterprise data

The whole pipeline is enclosed within Copilot, emphasising orchestration



Learning Language

Through training, the model implicitly learns:

- Grammar and syntax
- Semantics and meaning
- Long-range context dependencies
- Patterns across domains (science, fiction, instructions, etc.)



Beyond Pre-training

- **Feature extraction (repurposing)**

- Feature extraction, also known as repurposing, is a primary approach to fine-tuning LLMs. In this method, the pre-trained LLM is treated as a fixed feature extractor. The model, having been trained on a vast dataset, has already learned significant language features that can be repurposed for the specific task at hand.
- The final layers of the model are then trained on the task-specific data while the rest of the model remains frozen. This approach leverages the rich representations learned by the LLM and adapts them to the specific task, offering a cost-effective and efficient way to fine-tune LLMs.

- **Full fine-tuning**

- Full fine-tuning is another primary approach to fine-tuning LLMs for specific purposes. Unlike feature extraction, where only the final layers are adjusted, full fine-tuning involves training the entire model on the task-specific data. This means all the model layers are adjusted during the training process.
- This approach is particularly beneficial when the task-specific dataset is large and significantly different from the pre-training data. By allowing the whole model to learn from the task-specific data, full fine-tuning can lead to a more profound adaptation of the model to the specific task. Full fine-tuning requires more resources compared to feature extraction.





Generating Content with Deep Learning Networks

Generating content with a pretrained LLM

Tokenize the prompt

Convert text into tokens (numbers)

Embed tokens

Add meaning and position information

Apply attention

Focus on relevant tokens in context

Process with Transformer layers

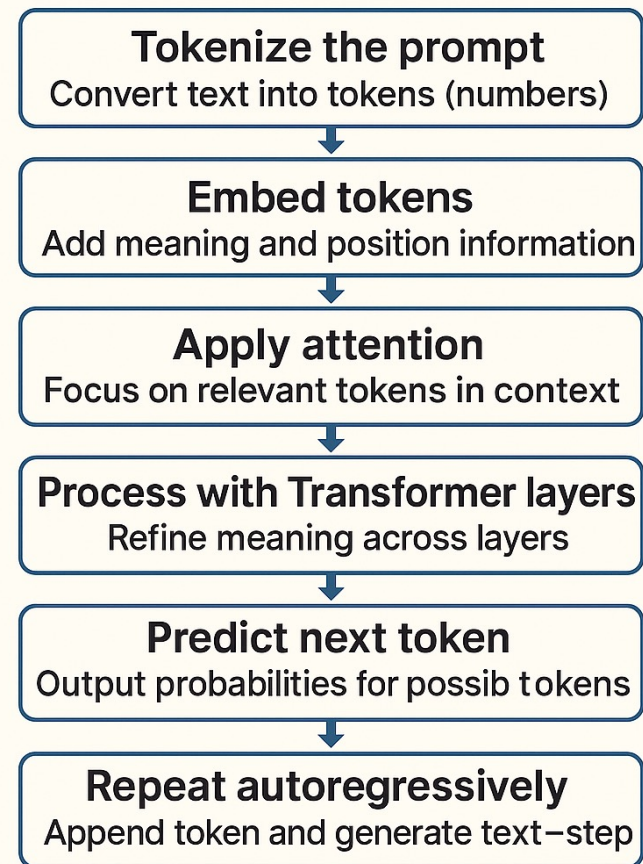
Refine meaning across layers

Predict next token

Output probabilities for possible tokens

Repeat autoregressively

Append token and generate text step-by-step



Input is tokenized

- The user provides input text (a prompt)
- The text is converted into tokens (numbers) using the same tokenizer used during training
- These tokens become the input sequence to the Transformer

Example:

“Large language models work by...”
→ [1987, 10234, 845, 563, ...]



Tokens are Embedded and Positioned

Inside the Transformer:

1. Each token is mapped to a vector embedding that represents its meaning
2. Positional encoding is added so the model knows:
 - word order
 - distance between tokens

This step gives the model both what the tokens are and where they occur.



Attention Mechanism Computes Token Relationships

The **self-attention mechanism** is the core of how Transformers work.

For each token, the model:

- Compares it with **all other tokens** in the input
- Computes **attention scores** indicating which tokens are most relevant
- Weights information from important tokens more heavily

This allows the model to:

- Track long-range dependencies
- Resolve ambiguity (e.g. what “it” refers to)
- Use context dynamically



Transformer Layers Refine Relationships

The input passes through multiple Transformer layers, each consisting of:

- Multi-head self-attention
- Feed-forward neural networks

Across these layers, the model builds:

- Rich semantic representations
- Higher-level abstractions of meaning

No learning occurs here — weights are fixed (pretrained).



Next token prediction

At the output of the Transformer:

The model computes a **probability distribution over the vocabulary**

Each possible next token has an associated probability

Example:

- "model" → 0.42
- "system" → 0.21
- "process" → 0.11



Sampling a token

The model selects the next token using a **sampling strategy**, such as:

- Greedy decoding
- Temperature scaling
- Top-k or top-p sampling

This balances:

- **Coherence** (safe, predictable tokens)
- **Creativity** (more varied outputs)



Autoregressive Generation Loop

The selected token is:

- Appended to the input sequence
- Fed back into the Transformer
- Used to predict the next token

This loop repeats until:

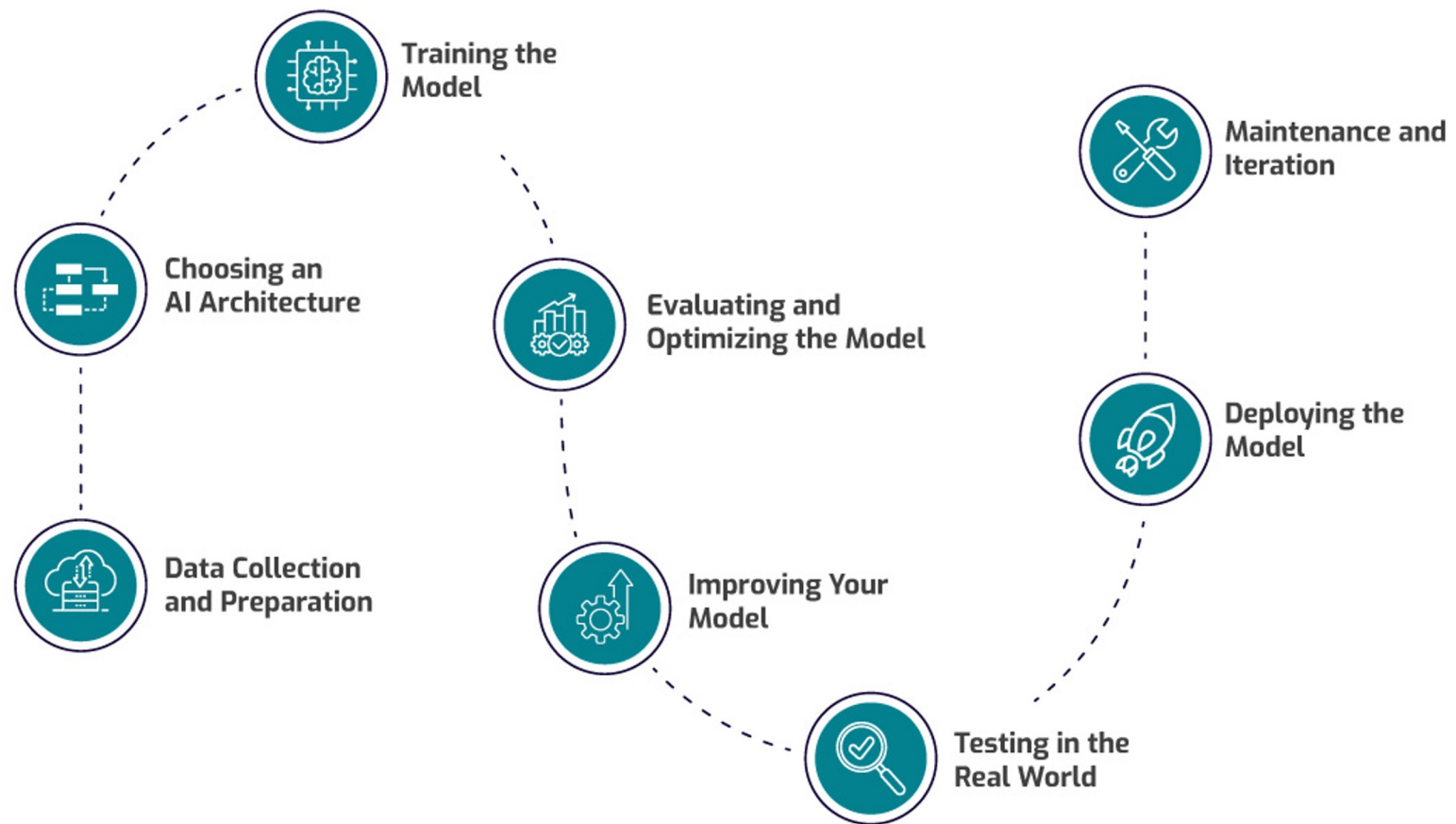
- A stop token is reached
- A maximum length is exceeded

This is why LLMs are called autoregressive models.





Deep Learning Models





Types of Generative AI Models

01

Generative Adversarial Networks (GANs)

Generative Adversarial Networks, or GANs, are considered the pioneer type of AI model introduced in 2014. Consisting of two parts named a generator and a discriminator, GANs have a high rate of successfully generating realistic content and finding applications such as art creation, cybersecurity, and data synthesis successful for AI training.

02

Large Language Models or LLM

Large Language Models (LLMs) are notable generative AI models, which have proven their efficiency in Natural Language Processing (NLP). LLM can be grouped into OpenAI's GPT and Google's BERT-similar groups. Trained on extensive text data, this model structures of generative AI apps efficiently performs tasks such as translating any data, summarizing, and producing human-like text.

03

Diffusion Models

Diffusion models can be called a recent innovation in generative AI models that have proven their performance in producing high-quality images and videos. Unlike traditional methods, diffusion models can improve image quality by refining the probability distribution of pixels. Its use in industries like photorealistic rendering, digital art creation, fashion design, and medical imaging is increasing day by day.

04

Variational Autoencoders (VAEs)

Variational autoencoders (VAEs) essentially combine elements of autoencoders and probabilistic models. They present input data in a simple format and then create new and innovative data models based on the presentation. Apart from generating output, this model also performs important tasks like image generation, data augmentation, and anomaly detection.

05

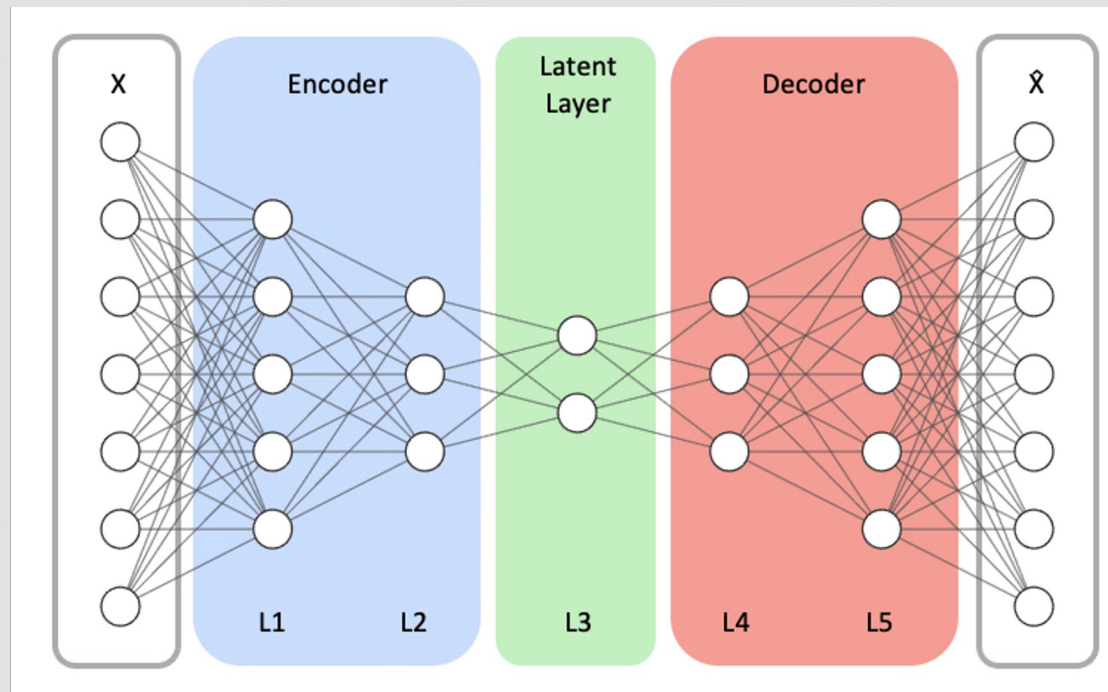
Transformer-based Models

Generative AI's model is powered by natural language processing and machine learning. So it can easily perform language translation, text generation, data retrieval, etc.

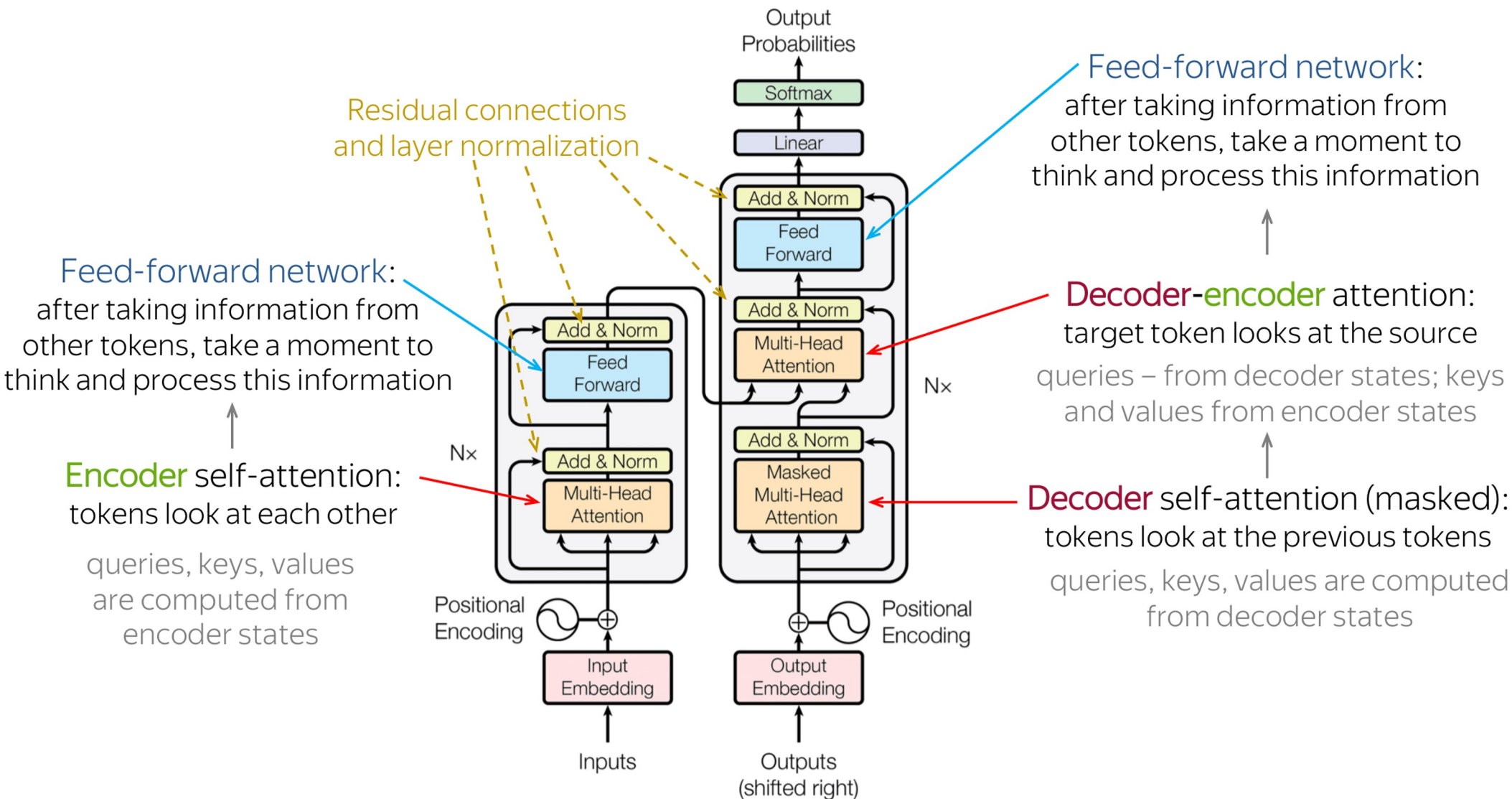
If you are able to select the perfect model for your Generative AI solution, it helps you to stand out from the crowd.



Autoencoder Neural Network



Song, Youngrok & Hyun, Sangwon & Cheong, Yun-Gyung. (2021). Analysis of Autoencoders for Network Intrusion Detection. *Sensors*. 21. 4294. 10.3390/s21134294.



GPT and BERT Transformer Models

GPT: Generative Pre-trained Transformer

- Autoregressive
- Each token can only see earlier tokens
- Trained using Next-Token Prediction
- Predicts the next token given all previous tokens
- **Goal: generate language**

BERT: Bidirectional Encoder Representations from Transformers

- Fully bidirectional
- Every token attends to all other tokens
- Trained using Masked Language Modelling
- Random tokens are hidden, and the model predicts the missing words
- **Goal: understand language**

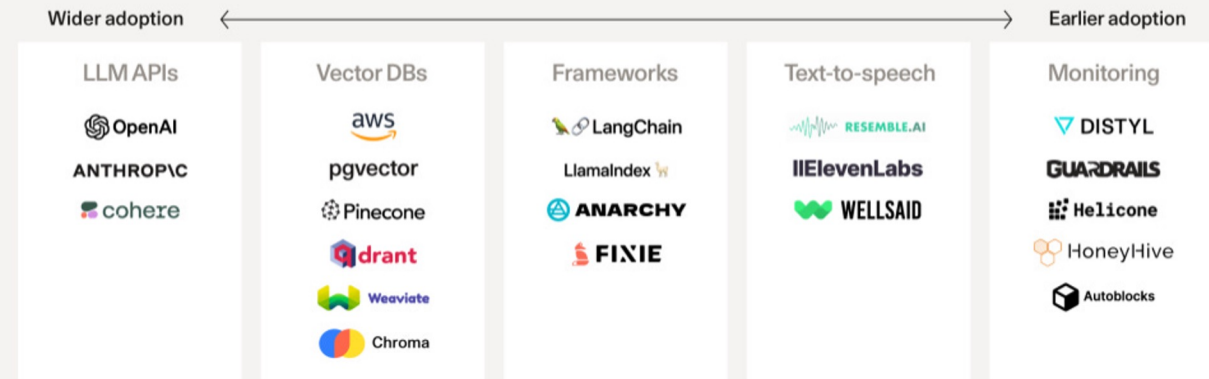


There are many companies involved in various steps of the development process

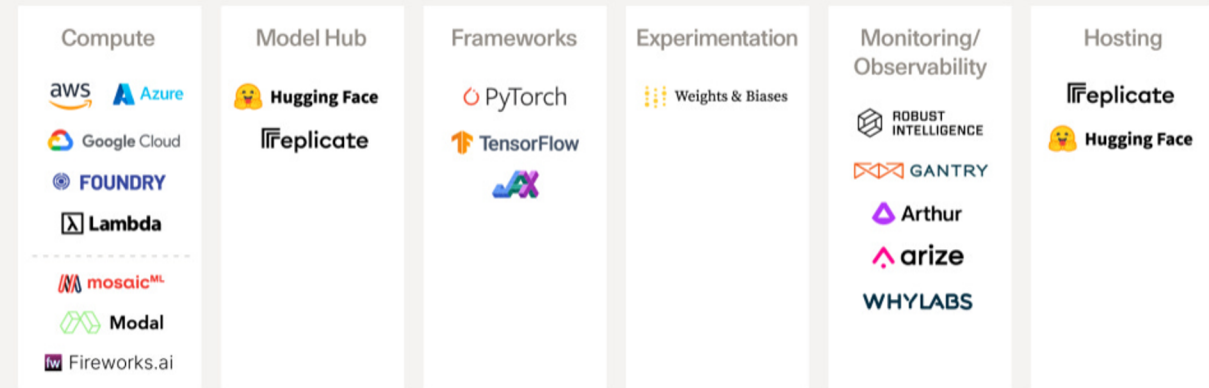
Retrieved from
<https://venturebeat.com/ai/inside-the-race-to-build-an-operating-system-for-generative-ai>



LLM API Stack



Custom Model Training / Tuning Stack



Note: this is not exhaustive. This is based on customer-reported usage and perception.

Transformer Model Implementations

- Google's Bidirectional Encoder Representations from Transformers was one of the first LLMs based on transformers.
- OpenAI's GPT followed suit and underwent several iterations, including GPT-2, GPT-3, GPT-3.5, GPT-4 and ChatGPT.
- Meta's Llama achieves comparable performance with models 10 times its size.
- Google's Pathways Language Model generalises and performs tasks across multiple domains, including text, images and robotic controls.
- Open AI's Dall-E creates images from a short text description.
- The University of Florida and Nvidia's GatorTron analyses unstructured data from medical records.
- DeepMind's Alphafold 2 describes how proteins fold.
- AstraZeneca and Nvidia's MegaMolBART generates new drug candidates based on chemical structure data.



Metric/Model	BERT	GPT	LLaMA4	Gemini	XLNet
Developer	Google	OpenAI	Facebook AI	Google	Google/CMU
Training Data Size	Large (Wikipedia, BookCorpus)	Large, maybe like 45TB of information	Large	Colossal (C4)	Large (diverse sources)
Training Data Source	Pre-Training and Fine Tuning	Pre-Training	Pre-Training and Fine Tuning	Pre-Training and Fine Tuning	Pre-Training
Architecture	Encoder only	Decoder-only	Encoder-Decoder Transformer	Encoder-Decoder Transformer	Encoder and decoder
Context Understanding	Bidirectional	Unidirectional (Contextual)	Bidirectional	Bidirectional	Bidirectional, more dynamic
Training Approach	Masked LM, Next Sentence Prediction	Reinforcement Learning, Supervised Learning, Unsupervised Learning	Masked LM, no NSP, longer training	Denoising Autoencoder	Masked LM
Primary Application	Language Processing, NLP tasks	Text generation, Creative writing	Multimodal foundation model	Multimodal foundation model	Language Processing, NLP tasks
Performance on Benchmarks	High	Varied, excels in creativity	Broad capability	Broad applicability	Varied
Fine-tuning	Required for	Required for specific	Highly adaptable	Highly adaptable	Requires careful





Survey for this Lecture